

Code Explanation

DLT Pipeline Code Explanation

The given code is a Delta Live Tables (DLT) pipeline written in Python, utilizing PySpark for data processing. It is designed to read data from CSV files, clean and transform the data, and then create summary tables using the transformed data.

Overview of the Code

This DLT pipeline is structured into several steps that read customer and order data from CSV files, clean and transform the data, and then create two summary tables: one for order summaries with Slowly Changing Dimension (SCD) Type 2 and another for aggregated customer spending. The pipeline is designed to handle data incrementally and maintain data history.

Step 1: Reading Source Data

This step involves reading customer and order data from CSV files stored in specified paths.

```
@dlt.table(
    name="customer_bronze",
    comment="Raw customer data loaded from CSV"
)
def customer_bronze():
    return spark.read.format("csv")
        .option("header", "true")
        .option("inferSchema", "true")
        .load(CUSTOMER_PATH)

@dlt.table(
    name="order_bronze",
    comment="Raw order data loaded from CSV"
)
def order_bronze():
    return spark.read.format("csv")
        .option("header", "true")
        .option("inferSchema", "true")
        .load(ORDER_PATH)
```

Step 2 & 4: Cleaning Customer Data

This step involves filtering out null values and removing duplicate records from the customer data.

```
@dlt.table(
    name="customer_silver",
    comment="Cleaned customer data with nulls and duplicates removed"
)
def customer_silver():
    return dlt.read("customer_bronze")
        .filter(
            col("CustId").isNotNull() &
            col("Name").isNotNull() &
            col("EmailId").isNotNull() &
            col("Region").isNotNull()
```

```
).dropDuplicates()
```

Step 3 & 4: Cleaning Order Data and Calculating Total Amount

This step involves filtering out null values, removing duplicate records, and calculating the total amount for each order.

```
@dlt.table(
    name="order_silver",
    comment="Cleaned order data with TotalAmount calculated and nulls/duplicates removed"
)
def order_silver():
    return dlt.read("order_bronze")
        .filter(
            col("OrderId").isNotNull() &
            col("ItemName").isNotNull() &
            col("PricePerUnit").isNotNull() &
            col("Qty").isNotNull() &
            col("Date").isNotNull() &
            col("CustId").isNotNull()
        ).dropDuplicates()
        .withColumn("TotalAmount", col("PricePerUnit") * col("Qty"))
```

Step 6-8: Creating SCD Type 2 Table for Order Summary

This step involves joining the cleaned customer and order data, and creating an SCD Type 2 table to maintain data history.

```
@dlt.table(
    name="ordersummary",
    comment="SCD Type 2 table with customer and order data",
    table_properties={
        "delta.enableChangeDataFeed": "true"
    }
)
@dlt.expect_or_fail("valid_customer_id", "CustId IS NOT NULL")
def ordersummary():
    # Get the current data from the silver tables
    customer_df = dlt.read("customer_silver")
    order_df = dlt.read("order_silver")

    # Join customer and order data
    joined_df = customer_df.join(order_df, "CustId", "inner").select
    (
        customer_df["CustId"],
        customer_df["Name"],
        customer_df["EmailId"],
        customer_df["Region"],
        order_df["OrderId"],
        order_df["ItemName"],
        order_df["PricePerUnit"],
        order_df["Qty"],
        order_df["Date"],
        order_df["TotalAmount"]
    )
```

```
)  
# ... rest of the code for handling SCD Type 2 logic
```

The SCD Type 2 logic involves checking for existing data, identifying changed records, marking them as inactive, creating new active records for changed data, and handling completely new records.

Step 9-10: Creating Customer Aggregate Spend Table

This step involves aggregating customer spending by name and date from the order summary data.

```
@dlt.table(  
    name="customeraggregatespend",  
    comment="Aggregated customer spending by name and date"  
)  
def customeraggregatespend():  
    return dlt.read("ordersummary")  
        .filter(col("IsActive") == True)  
        .groupBy("Name", "Date")  
        .agg({"TotalAmount": "sum"})  
        .withColumnRenamed("sum(TotalAmount)", "TotalAmount")  
        .select("Name", "TotalAmount", "Date")
```